

iskHED: An Automated Scheduling System for Polytechnic University of the Philippines - College of Computer Information Science (PUP CCIS) Using Hybrid Optimization Framework

1. Introduction and its Background

The accelerating pace of digital transformation has reshaped every dimension of life, from how organizations manage resources to how individuals navigate complex environments. Nowhere is this shift more consequential than in higher education. There is a convergence of growing student populations, constrained physical infrastructure, and increasingly complex regulatory requirements that places enormous pressure on academic planning systems.

In the context of the Philippines, the Commission on Higher Education (CHED) has consistently called on State Universities and Colleges (SUCs) to modernize their workflow with technology-driven solutions, and national frameworks. One example of such a framework is the Digital Transformation of Philippine Higher Education report which underscored the urgency of this transition (World Bank, 2022). Mobile and web applications have emerged as indispensable tools in this effort, enabling institutions to automate complex tasks that once required significant manual labor and were prone to human error (Lagyo et al., 2025; Ma, 2025).

Class scheduling is one of the most resource-intensive and conflict-prone administrative tasks in Philippine higher education. Schedule creation is predominantly manual across SUC's, taking a week or more and involving registrars and program chairpersons who must simultaneously satisfy room availability, faculty loads, modality requirements, and student sectioning (Romaguera et al., 2024).

Yee and their associates (2025) found that traditional scheduling methods on the City College of Angeles relies on manual or heuristic-based approaches which systematically produce inefficiencies and scheduling conflicts. These problems compound at high-density institutions, where the volume of sections, the variety of room types, and the mix of face-to-face and online modalities make manual coordination increasingly untenable. Mallari, San Juan, and Li (2023), in their Philippine-authored study published in *Computers and Industrial Engineering*,

further document that university scheduling is typically performed arbitrarily and with little coordination among department chairs and faculty members, leading to avoidable resource conflicts and uneven workload distribution.

The Philippine scheduling problem has lasting problems beyond administration. This inadequacy has direct and measurable consequences for students. Poor time management is closely associated with elevated stress and reduced academic performance in university populations (Soares et al., 2023; Bangquiao et al., 2023). When schedules are poorly designed, they tend to have long unexplained gaps, abrupt modality switches, or unresolved room conflicts. Students thus have to bear the cognitive and logistical cost of such design issues.

Bangquiao et al. (2023) documented the coping strategies used by working students at Philippine universities, including calendar blocking and selective multi-tasking, as adaptive responses to an institutional scheduling environment that fails to account for their circumstances. Landon and Pope (2025) further demonstrated that abrupt modifications to institutional calendars trigger measurable increases in student stress and performance degradation, reinforcing the psychological stakes of scheduling quality.

Within this broader context, the Polytechnic University of the Philippines College of Computer Information Science (PUP CCIS) faces a distinctive and acute version of these challenges. The campus is large, multi-floor, and operates under a hybrid modality system that intermixes face-to-face and online classes within the same student schedule. Academic schedules at CCIS are frequently unpredictable: sudden class suspensions, last-minute room reassignments, and abrupt modality shifts leave students with limited information and no systematic tool to reorganize their day.

Currently, students must manually cross-reference their class schedules when planning their week, a fragmented and error-prone process (Lagyo et al., 2025). While general-purpose navigation applications exist, they lack the granular indoor-outdoor path awareness needed for a dense Philippine academic campus and carry no awareness of class schedules, room-level occupancy, or institutional rules (Lagyo et al., 2025; Ma, 2025). Ceccarini et al. (2021) argued for precisely this kind of integration, recommending smart campus architectures that link spatial mapping directly with classroom occupancy and real-time scheduling data.

The problems currently experienced by PUP CCIS students span several interconnected dimensions:

- *Lack of Optimized Routing:* Mainstream navigation applications prioritize vehicular flow and are unable to provide granular, pedestrian-focused indoor-outdoor routing across dense Philippine campuses (Lagyo et al., 2025).
- *Unproductive Idle Time:* Extended gaps between classes or sudden cancellations leave students without a structured plan. Standard GPS technologies do not function effectively indoors and cannot surface localized real-time schedule data (Ma, 2025).
- *Difficulty in Adjusting:* Students find it cognitively demanding to reorganize their academic day when schedules become unpredictable due to unforeseen changes (Bangquiao et al., 2023).
- *Time Management and Wellbeing:* Poor time management is closely associated with elevated stress and reduced academic performance (Soares et al., 2023; Bangquiao et al., 2023). A systematic review on smart campus technologies confirms that a university's technological environment measurably shapes student experience and engagement, and that systems designed around user needs actively support rather than hinder academic outcomes (Zhang et al., 2022; Ceccarini et al., 2021). Well-designed scheduling tools are a direct expression of this user-centered principle.

Addressing these challenges requires a system capable of both enforcing hard institutional rules and adaptively optimizing complex scheduling variables. The literature consistently identifies hybrid algorithmic approaches as the most effective solution for this class of problem. A Constraint Satisfaction Problem (CSP) framework enforces hard rules such as preventing room double-bookings and ensuring no professor is assigned to two simultaneous classes (Bulawat & Pena, 2025). The Most Constrained Variable (MCV) heuristic, also documented as the Minimum Remaining Values (MRV) heuristic, operates within the CSP layer to prioritize scheduling decisions for the most tightly constrained courses first, reducing search space complexity and minimizing backtracking (Russell & Norvig, 2022).

A Genetic Algorithm (GA) then evolves populations of candidate schedules over successive generations until an optimized layout emerges (Agoylo et al., 2025; Romaguera et al., 2024). To finalize a schedule, Simulated Annealing (SA) refines the output by accepting occasional suboptimal moves early in its process, escaping local optima before gradually converging on a near-optimal final schedule (Sylejmani et al., 2023).

This study therefore aims to develop iskHED: an automated scheduling system for PUP CCIS that integrates CSP with MCV ordering, GA-based population evolution, and SA-based refinement into a unified pipeline. The goal is to produce optimal, conflict-free academic schedules that fully comply with CHED and PUP regulations while respecting the practical constraints of the CCIS campus environment.

2. Review of Related Literatures

This section examines studies that inform the system design methods and algorithmic frameworks employed by iskHED. With PUP CCIS serving a high-density student population across multiple year levels and subject categories, the institution faces scheduling challenges that are simultaneously logistical, spatial, and psychological found in local and international papers.

Scheduling Challenges in Philippine Higher Education

Manual class scheduling has long been documented as a systemic weakness across Philippine state universities and colleges. Yee et al. (2025) conducted a study at the City College of Angeles and found that traditional scheduling methods relying on manual or heuristic-based approaches consistently produced inefficiencies and scheduling conflicts, motivating the development of an interactive timetabling matrix to address the problem. Their findings are broadly consistent with conditions at other Philippine SUCs: scheduling is performed by registrars and program chairpersons working from spreadsheets, a process that is time-consuming, error-prone, and difficult to scale as institutions grow.

Camungao (2025) documented that manual schedule crafting at Isabela State University takes a week or more and routinely produces overlapping subjects, time conflicts in rooms and laboratories, and uneven distribution of faculty workloads. The downstream effects are directly instructional: faculty members miss

scheduled classes due to unresolved conflicts, topics go undiscussed within the semester, and students receive inadequate preparation as a result. These findings from a Philippine SUC context establish that the scheduling problem iskHED addresses is not unique to PUP CCIS but is a documented institutional pattern across the Philippine public higher education system.

Romaguera, Plender-Nabas, Matias, and Austero (2024), in a study published in *Procedia Computer Science*, developed a web-based timetabling system for Caraga State University using an enhanced genetic algorithm. Their system demonstrated that automated scheduling not only resolves hard constraint conflicts more reliably than manual methods but also satisfies soft constraints (such as minimizing room switches and balancing workloads) that manual processes routinely neglect. Their use of real institutional datasets from a Philippine SUC provides direct empirical grounding for iskHED's design objectives.

Mallari, San Juan, and Li (2023) from De La Salle University, publishing in *Computers and Industrial Engineering*, formalized the University Coursework Timetabling Problem and proposed an optimization model that improved timetable quality by 9.45% over manual methods. They documented that scheduling in Philippine university settings is typically performed without formal coordination among decision-makers, leading to avoidable conflicts and suboptimal resource use. Garcia (2022), as mentioned by Keh and Sarmiento (2025), further examined the challenges of implementing automated systems in Philippine higher education, concluding that streamlined scheduling directly fosters a more conducive learning environment and supports educators in managing their teaching responsibilities.

Constraint Satisfaction Problem (CSP)

A Constraint Satisfaction Problem (CSP) is a formal framework in which the goal is to assign values to a set of variables such that all defined constraints are simultaneously satisfied. As formalized by Russell and Norvig (2022), a CSP is defined by three components: a set of variables, a domain of possible values for each variable, and a set of constraints that restrict the permissible combinations of values. In university timetabling, variables represent courses or sections, domains represent the set of permissible time slots and rooms, and constraints encode institutional rules — such as the requirement that no professor or room is assigned to two simultaneous events. The scheduling engine constructs a valid timetable by

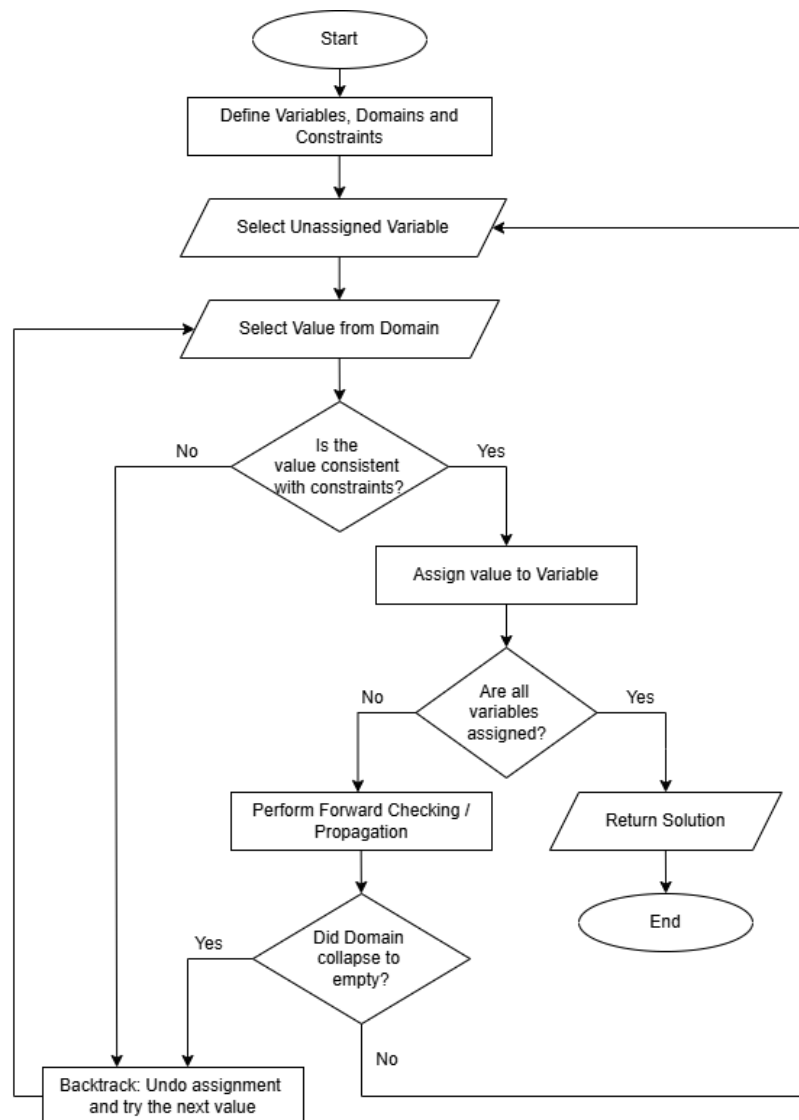
systematically searching this domain and assigning values without violating any constraint.

Bulawat and Pena (2025) applied a partial ADDIE framework to develop an automated scheduling system for a Philippine off-campus institution, using systematic needs analysis and structured blueprinting to ensure the system's constraints accurately reflected verified room capacities and faculty loads. Their findings confirm that grounding hard constraints in real institutional data before any optimization algorithm runs significantly reduces invalid schedule outputs. This approach directly informs iskHED's design: CCIS-specific rules (i.e. room locations, modality policies, faculty load limits) are encoded into the CSP layer before GA or SA operates.

Ballera and Napalit (2023) further demonstrated the value of embedding hard constraint checking directly within a scheduling engine, showing that this approach significantly shortens processing time and prevents overlapping assignments under high-density scheduling conditions. Agoylo et al. (2025), in building a web-based scheduling system combining GA with biometric verification, reinforced that even population-based optimizers depend critically on a reliable constraint layer to prevent invalid solutions from propagating across generations/ This establishes CSP as the structural foundation on which all other optimization algorithms in iskHED's pipeline operate.

Figure 1

The diagram showcases the Constraint Satisfaction Problem (CSP) validation loop algorithm (W3Schools, 2026). This will be used to block room double-bookings and prevent conflicting professor assignments in the Administrative Master Scheduling Module.



Most Constrained Variable (MCV) Heuristic

The Most Constrained Variable (MCV) heuristic, also known as the Minimum Remaining Values (MRV) heuristic, is a dynamic variable ordering strategy used within CSP backtracking search. Russell and Norvig (2022) formally define MCV as

the strategy of selecting the unassigned variable with the fewest remaining legal values in its domain at each decision point, characterizing it as a “fail-first” approach: by prioritizing the variable most likely to produce a conflict, the algorithm detects dead-ends early and prunes large portions of the search tree before they are explored, dramatically reducing backtracking.

Empirical benchmarks consistently confirm this efficiency gain. Implementations pairing MCV with arc-consistency algorithms such as AC-3 have been shown to require up to 45% fewer operations to solve equivalent scheduling problems compared to uninformed variable ordering (Stanford CS221, 2023). A Python-implemented CSP scheduling system tested on real-world university data, encompassing 200 courses, 45 instructors, and 20 rooms, demonstrated that combining MRV with degree heuristic tie-breaking produced a high feasibility rate and zero hard-constraint violations (IAJSE, 2025).

In iskHED, MCV operates as the intelligent ordering layer within the CSP module. Rather than assigning time slots arbitrarily, the system first identifies the courses with the fewest remaining legal assignment options (such as laboratory classes locked to the 5th Floor South Wing with a face-to-face-only modality requirement) and resolves these before more flexible courses consume their only viable slots. This prevents the costly backtracking that would arise from scheduling flexible courses first, and sets up a well-formed, constraint-compliant population for the Genetic Algorithm to evolve.

Genetic Algorithm (GA)

A Genetic Algorithm (GA) is a population-based optimization technique in which a set of candidate solutions, each encoded as a chromosome representing a complete schedule, evolves over successive generations toward an optimal layout. As implemented in Philippine timetabling research, the algorithm evaluates each solution using a fitness function measuring compliance with hard and soft constraints, selects high-performing solutions as parents, and applies crossover operators to combine parent traits and mutation operators to introduce controlled variation that prevents premature convergence (Agoylo et al., 2025; Romaguera et al., 2024).

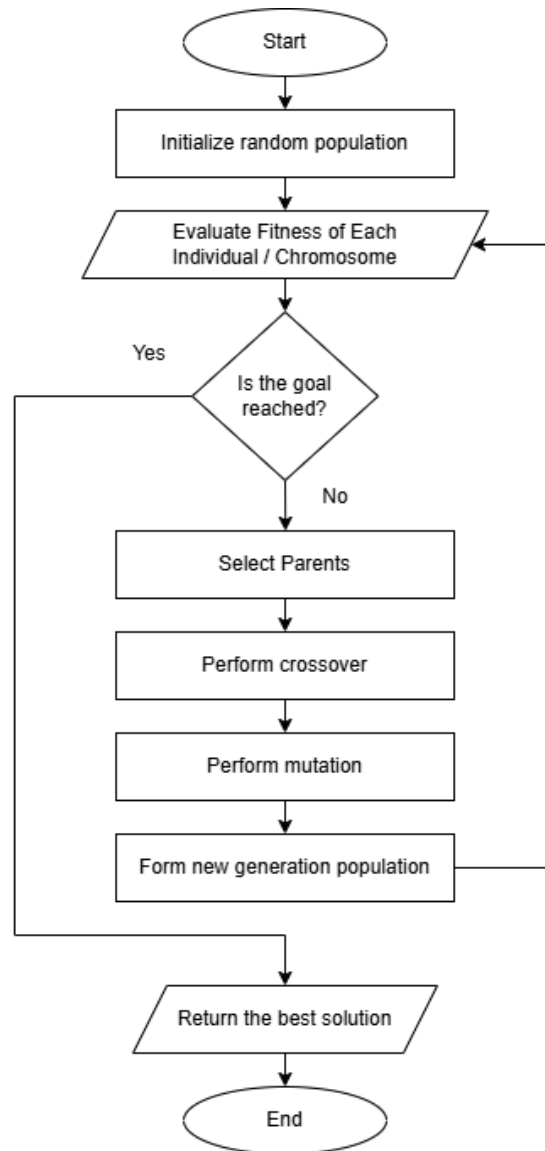
Within the Philippine context, Romaguera et al. (2024) developed and deployed an enhanced GA-based timetabling system for Caraga State University,

published in *Procedia Computer Science*. Their enhancement introduced a heuristic mutation operator that specifically targeted infeasible genes, improving the algorithm's capacity to explore and exploit the solution space. Using real institutional datasets, their system optimized classroom resource use by minimizing the number of rooms required while satisfying both hard and soft constraints — a direct analogue to CCIS's room allocation challenge across the East Wing, South Wing, and Gymnasium.

Agoylo et al. (2025) further validated GA's applicability to Philippine SUC scheduling by deploying a web-based system combining population-based genetic algorithms with biometric verification. Their results demonstrated that GA successfully resolved multi-layered scheduling conflicts across successive generations, producing balanced academic days while maintaining platform stability under real institutional conditions. Internationally, a hybrid POGA-DP system combining GA with dynamic programming demonstrated improved computational efficiency through judgment-based exchange operations and forced repair mutations (MDPI Algorithms, 2025), confirming that GA performance can be further enhanced when paired with complementary refinement mechanisms such as Simulated Annealing.

Figure 2

The diagram showcasing Genetic Algorithm (GA) evolutionary lifecycle (W3Schools, 2026). This will be used to generate complete, resource-efficient semester schedule populations.



Simulated Annealing (SA)

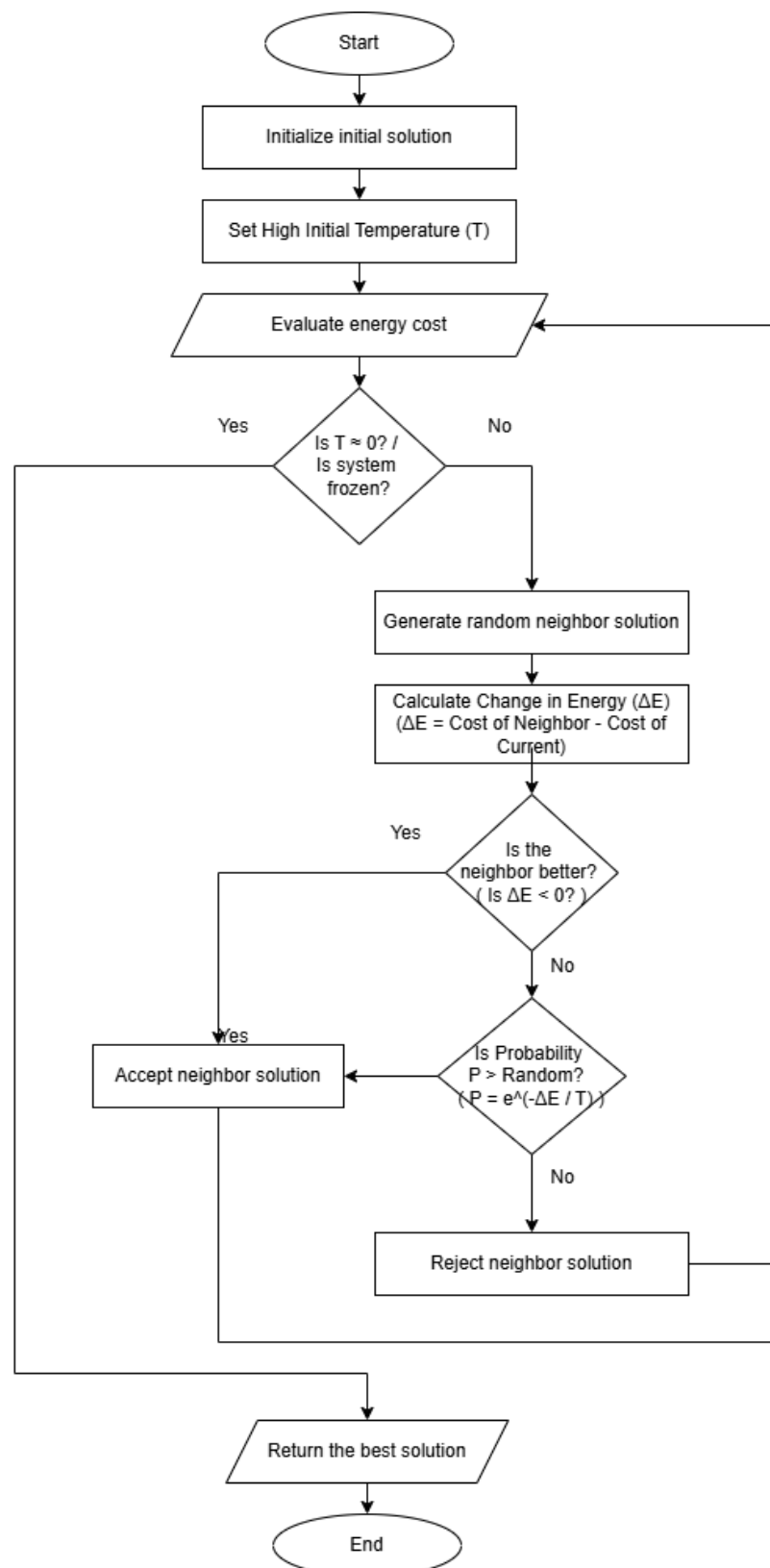
Simulated Annealing (SA) is a probabilistic metaheuristic optimization algorithm that draws on the physical analogy of controlled material cooling. As described by Sylejmani et al. (2023), the algorithm begins at a high “temperature” parameter that allows it to probabilistically accept neighbor solutions worse than

the current state, enabling aggressive exploration of the search space and escape from local optima. As the temperature decreases according to a predefined cooling schedule, this tolerance narrows and the algorithm gradually converges on a near-globally-optimal solution. The acceptance probability for a worse solution is governed by the Boltzmann function $P = e^{(-\Delta E/T)}$, where ΔE is the cost difference and T is the current temperature.

Sylejmani et al. (2023) developed a two-phase SA solver for the University Timetabling Problem that simultaneously handles complex constraints including student sectioning, room assignment, and distribution rules. Their results demonstrated that SA's graduated penalization mechanism allowed the system to escape search dead-ends and converge on near-conflict-free timetables even across highly diverse institutional configurations — a finding directly relevant to CCIS's mixed modality, multi-floor, multi-year-level scheduling context. In iskHED, SA operates as the final refinement stage, receiving the GA-evolved schedule population and iteratively fine-tuning it: balancing class gaps, minimizing modality switches, and distributing workloads equitably across students and faculty.

Figure 3

The diagram showcasing Simulated Annealing (SA) metaheuristic optimization algorithm (W3Schools, 2026). This will be used to iteratively refine schedule drafts, balance class gaps, and distribute student workloads.



Structural and Operational Framework

Bulawat and Pena (2025) provided a detailed blueprint for implementing a partial ADDIE framework in automated academic scheduling systems, emphasizing systematic upfront needs assessment and constraint verification before any algorithm runs. Their approach ensures the system's logic is grounded in real-world institutional data rather than assumed parameters. From an operational standpoint, Lu et al. (2022) measured the significant impact of schedule consistency and predictable operational patterns on human performance and burnout reduction, supporting iskHED's design emphasis on generating stable, balanced schedules that preserve student productivity.

The World Bank's (2022) Digital Transformation of Philippine Higher Education report identifies automated academic planning tools as a key mechanism for improving institutional efficiency and student outcomes across Philippine SUCs. This national framing positions iskHED not merely as a local departmental convenience but as a contribution to a broader institutional modernization agenda that CHED and the national government have formally prioritized.

Psychological Perspective on Class Scheduling

Landon and Pope (2025) investigated the psychological impact of abrupt modifications to standardized institutional calendars, documenting measurable increases in student stress and performance degradation when schedules change unpredictably. Their findings support the inclusion of defensive stability features in iskHED — specifically, the SA layer's soft constraint optimization for minimizing unnecessary gaps and modality switches, which reduces the cognitive burden of daily schedule navigation.

Research on smart campus technologies confirms the link between a university's digital infrastructure and the quality of student experience. Zhang and their co-authors (2022), in a systematic review of smart campus applications published in IEEE Access, documented that campus systems designed around user needs rather than technology for its own sake produce measurably better outcomes for students and faculty alike.

Ceccarini and associates (2021) similarly demonstrated that integrating IoT and data visualization into campus infrastructure enhances sustainability and user engagement, reinforcing the case for user-centered design in academic systems.

Bai and their collaborators (2022) further demonstrated that users perceive automated scheduling systems as more objective and fair than human-assigned schedules, particularly when distribution outcomes are unfavorable. This supports iskHED's use of algorithmic allocation as a transparent, rule-based alternative to manual processes.

Habit and Demographic Perspective on Students

Holili et al. (2024) identified significant operational friction between heavy institutional workloads and individual student time management structures, pointing to the need for integrated deadline and schedule indicators that prevent academic plans from collapsing under pressure. Soares et al. (2023) similarly documented the progressive erosion of student self-discipline when faced with uncoordinated, overlapping project deadlines. This further enforces iskHED's use of synchronized schedule tracking as a primary feature of the student-facing interface.

Bangquiao et al. (2023) analyzed the coping strategies used by working students at Philippine universities, including calendar blocking and selective multi-tasking, to maintain academic balance under time pressure. Their findings directly inform iskHED's dedicated time-blocking module for working students, which treats irregular scheduling not as an edge case but as a first-class feature of the system's design.

3. Objectives

General Objective

- To build a flexible, automated class scheduling system for Polytechnic University of the Philippines – College of Computer Information Science (PUP CCIS) that combines deterministic and non-deterministic methods to generate optimal, conflict-free academic schedules for both students and faculty.

Specific Objectives

This study aims to do the following:

- Design and implement a Constraint Satisfaction Problem (CSP) module that enforces hard scheduling constraints, including prevention of room double-bookings, avoiding overlaps in professor assignment, and all CHED and PUP rules on modality, class hours, and room assignments are followed.
- Create a Most Constrained Variable (MCV) -based engine for pathfinding and immediate task assignment to be used to calculate the shortest walks between buildings and fill unexpected schedule gaps with important periods.
- Integrate a Genetic Algorithm (GA) that evolves several schedule populations over many generations, using selection, crossover, and mutation operators while also checking their fitness based on hard constraints.
- Use Simulated Annealing (SA) to iteratively improve timetables, escape local optima with controlled temperature-reductions, and deliver final schedules that minimize conflicts and distribute work fairly among students and faculty.
- Ensure optimal performance by testing its ability to generate constraint-compliant schedules matching with the current state of CCIS faculty and students for every year level and subject categories of the upcoming semester.

4. Scopes and Limitations

The proposed system is scoped in the context of the Polytechnic University of the Philippines - College of Computer Information Science (PUP CCIS). The system aims to form schedules throughout all year levels with subjects consisting of four categories: Physical Activities Toward Health and Fitness (PATHFIT), general education (GEED), lectures (LEC), and laboratories (LAB). Lectures and GEED classes are allocated on the 4th floor of the East Wing. Laboratory classes are held on the 5th floor of the South Wing, while PATHFIT classes are held in the gymnasium. This system is designed to be run by authorized personnel during the enrollment period.

The system employs an approximate approach utilizing both deterministic and non-deterministic algorithms to generate near-optimal schedules. While hard constraints are strictly enforced, soft constraints are optimized but not guaranteed

as shown on Table 1. The system accounts for forecasts and not real-time changes. Any unforeseeable modifications such as faculty absence require manual adjustment. Irregular student scheduling is handled separately through a secondary constraint-matching process. The accuracy of the generated schedule is also dependent on the completeness and correctness of the input data as shown on Figure 4. The system is only intended to assign time and day slots on classes and not distribute professors to blocks.

Figure 4

A conceptual map for the scheduling process that maps out the expected inputs, process, and outputs of the system.

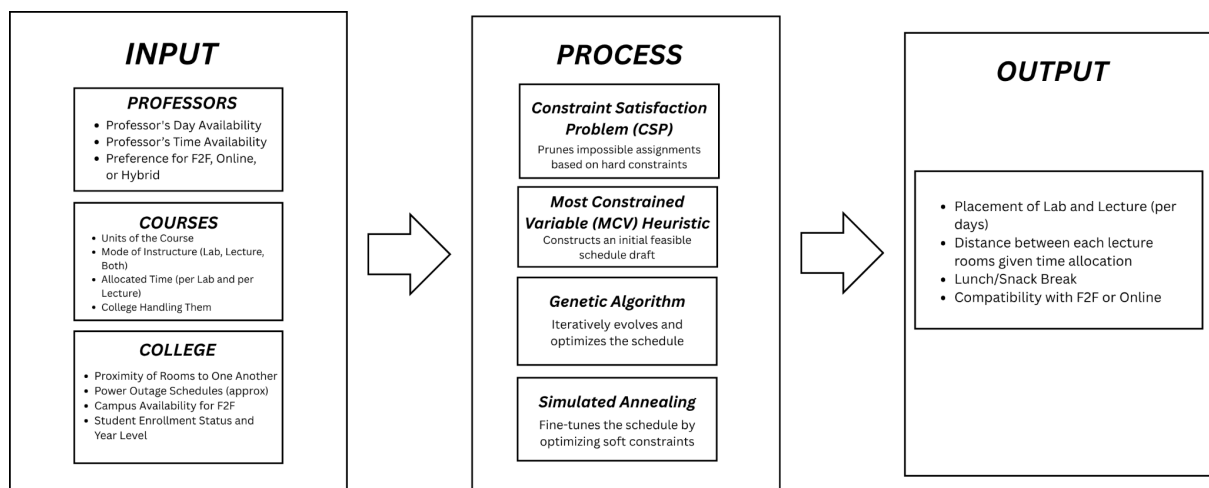


Table 1

A list of hard and soft constraints that are based on both the PUP Student Handbook and Commission on Higher Education articles. They are both going to be used for the creation of an initial schedule (hard constraints) and refining it (soft constraints).

Hard Constraints	Soft Constraints
No mix of modality UNLESS there is	Break must be given to each

a significant 3–4 hour gap between subjects — one subject per day, two instances in a week (CHED, 2022) • Three no-class days per week. Sunday is automatically a no-class day except for NSTP for first year students (PUP, 2019) • One unique time slot per block EACH professor (CHED, 2008) • Scheduler must account for a professor's class in ALL year levels (CCIS observed scheduling practice) • Schedule for F2F is from 8:00 AM to 8:00 PM while online classes can start from 7:30 AM to 9:00 PM (CCIS observed scheduling practice) • Locations are either at 4th Floor East Wing, 5th Floor South Wing, and the Gymnasium (CCIS observed scheduling practice) • Laboratory classes must be conducted face-to-face and cannot be moved online. In the event of a power outage, lecture classes must be relocated to South Wing or rescheduled (CHED, 2022) • Laboratory and lecture must be together	schedule, lasting at least 1 hour and extending up to 4 hours if a modality switch is required. • If the break skews at lunch time or afternoon, it must be longer. • No more than 4 consecutive hours of class without at least 1 hour break (CCIS observed scheduling practice) • For irregulars: All enrolled courses aligned to their year level get priority first, then courses being retaken are adjusted to those adjacent to them (PUP, 2019) • Lecture classes affected by power outage schedules may be allocated to South Wing classes if necessary, as these classes can proceed without presentation-dependent technology (CCIS observed scheduling practice)
--	---

5. System Design and Methodology

The proponents plan to implement a structured agile development cycle to iteratively design, build, and refine the scheduling platform architecture. The primary goal of this immediate development phase is to rapidly launch a functional

minimum viable product tailored specifically for administrative operators. By focusing execution on this administration prototype, the system can immediately deliver core functionality to manage foundational constraints while allowing for continuous enhancements based on operational feedback.

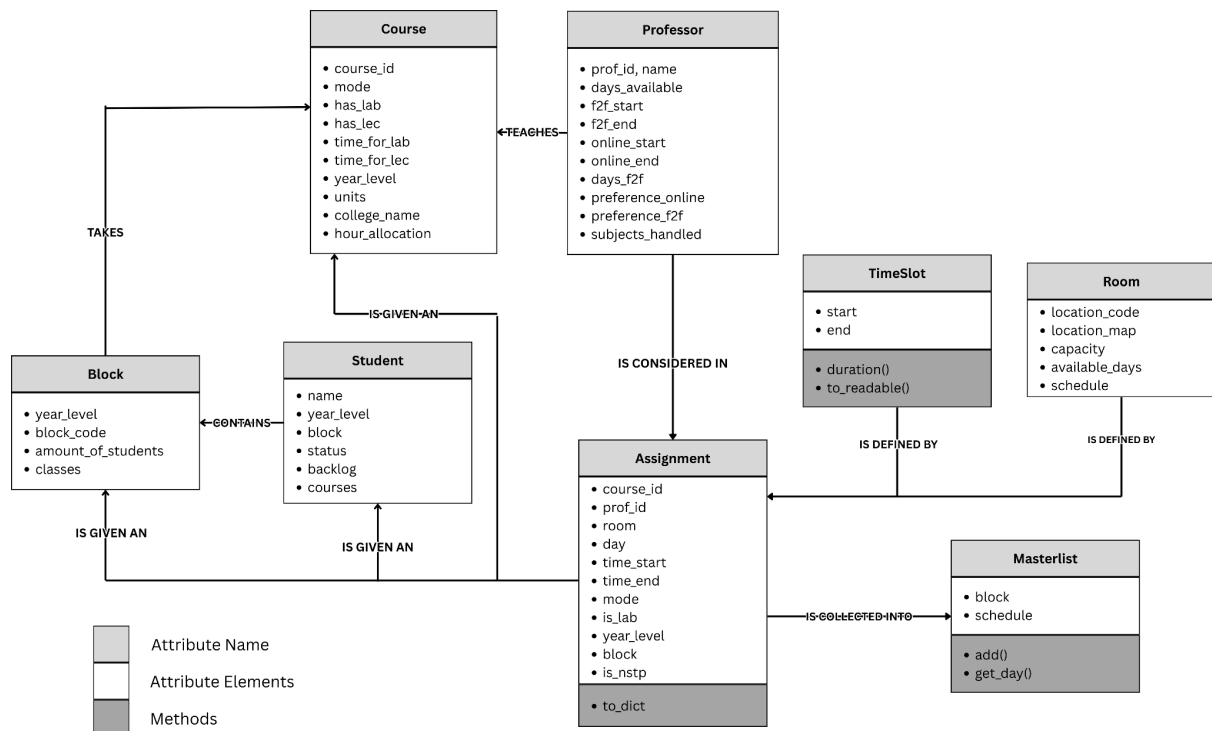


Figure 1. Architectural Class Diagram for the Automated Scheduling System

This figure demonstrates the structure of the scheduling system and displays the transformation of theoretical academic process into practical object oriented structures, resulting in the creation of an ultimate and physical object. This flow is based on the fact that Block specifies the initial Student group and identifies the entire Courses that need to be taken by this group. This will be used along with constraints imposed by Professors, Time Slots, and Rooms to generate the primary Assignment object, which is finally distributed to other organizational materials to form the Scheduling Masterlist.

NAME	DESCRIPTION
Assignment	The central operational entity that binds a Block, Student, Course, Professor, TimeSlot, and Room together into a concrete

	schedule entry.
Block	This defines a group of students at a specific year level. This serves as a primary container that holds students and determines the set of courses they must take together.
Course	Represents an academic subject containing details such as units, mode of delivery (lecture/lab), year level requirements, and total hour allocations. It links students to what they need to learn and professors to what they must teach.
Masterlist	The final combined schedule database that collects all individual assignments and structural data to generate a comprehensive, distributed timetable for the entire institution.
Professor	Represents the faculty member teaching the course. It contains constraints such as handling preferences (online vs. face-to-face), available days, and specific subjects handled.
Room	Represents the physical or virtual location where an assignment takes place, tracking parameters like capacity, location map, and its own individual availability schedule.
Student	Represents an individual learner that is enrolled in the system. A student belongs to a specific academic block, tracks their status, and is assigned a specific set of courses.
TimeSlot	Defines the operational window (start time, end time, and duration) available for academic activities to prevent scheduling conflicts.

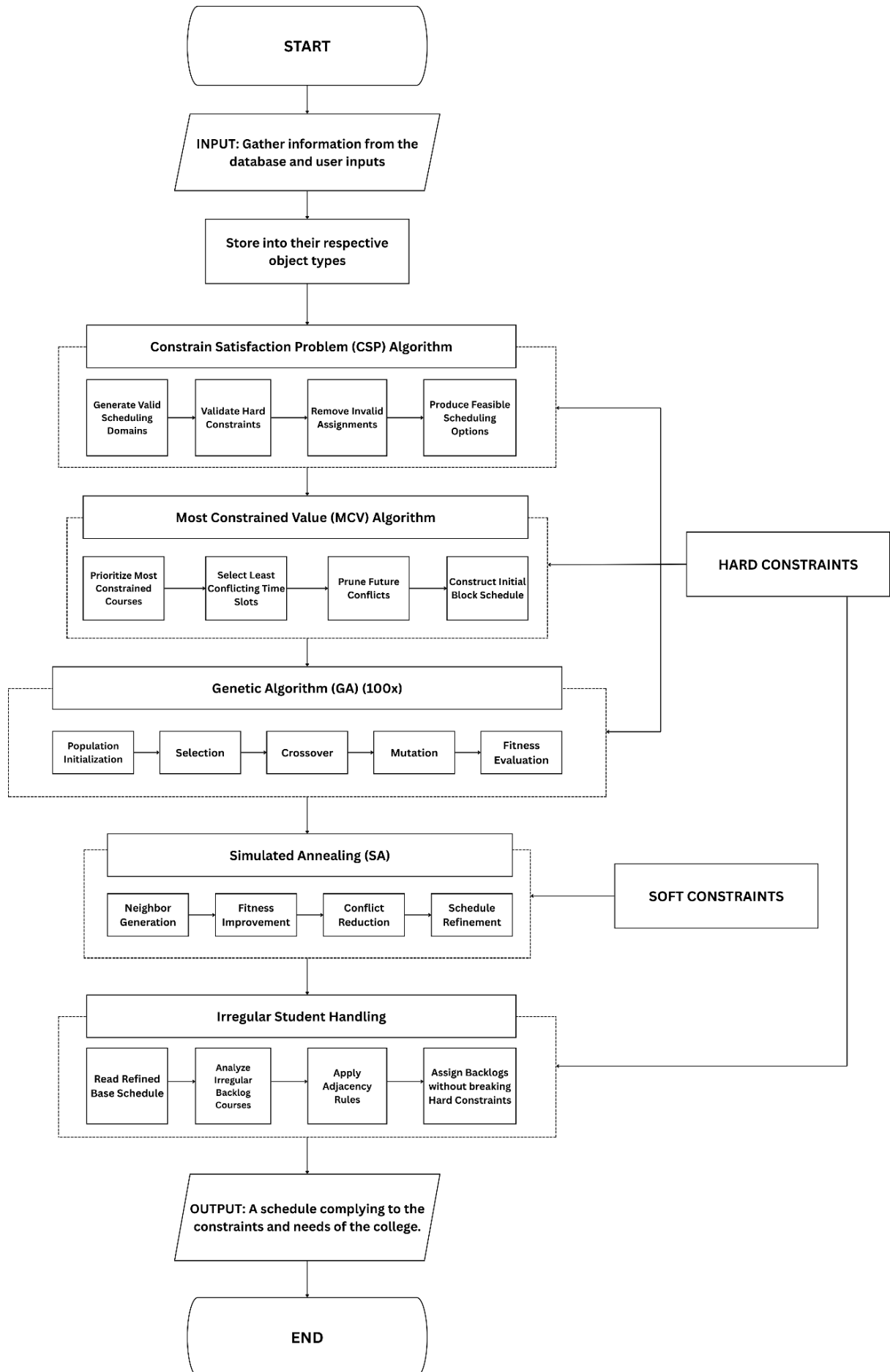


Figure 2. *Scheduling Flowchart with Irregular Student Resolution Process.*

The flowchart describes the overall algorithmic process starting from input and ending in output generation. Input data structures are then fed sequentially into a CSP algorithm and MCV algorithm to generate a viable schedule that meets all the hard constraints. The schedule is further improved using a combination of a GA loop and SA pass in order to reduce violations of soft constraints. Lastly, the irregular student subroutine of the system is run, which consists of reading the improved base schedule, checking for backlog courses, and employing adjacency rules dynamically to ensure the scheduling of irregular students without violating any structure.

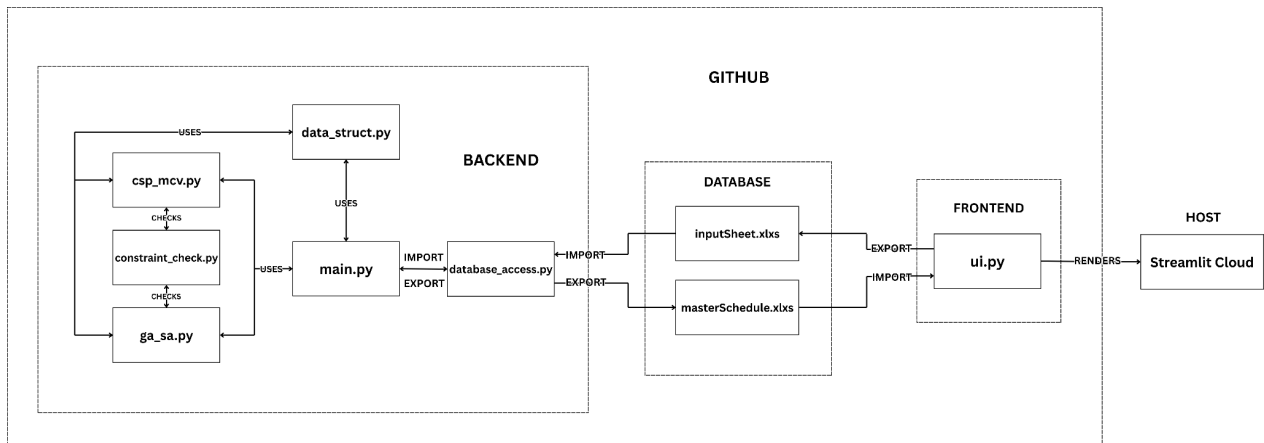


Figure 3. *System Architecture and Deployment Pipeline of the Minimum Viable Product (MVP) Scheduling Prototype*

The above diagram shows the flow of the lightweight architecture used to develop a highly adaptable and maintainable minimum viable product hosted on the web. It shows the interaction between the backend subsystem that consists of distinct computational algorithms converging in main.py through the file-based database system, which stores user input logs and schedules created. This is done directly from the Streamlit frontend layer. The whole process is governed using GitHub for version control and automatically deployed to Streamlit Cloud.

	Filename	Description	Libraries Being Used
--	----------	-------------	----------------------

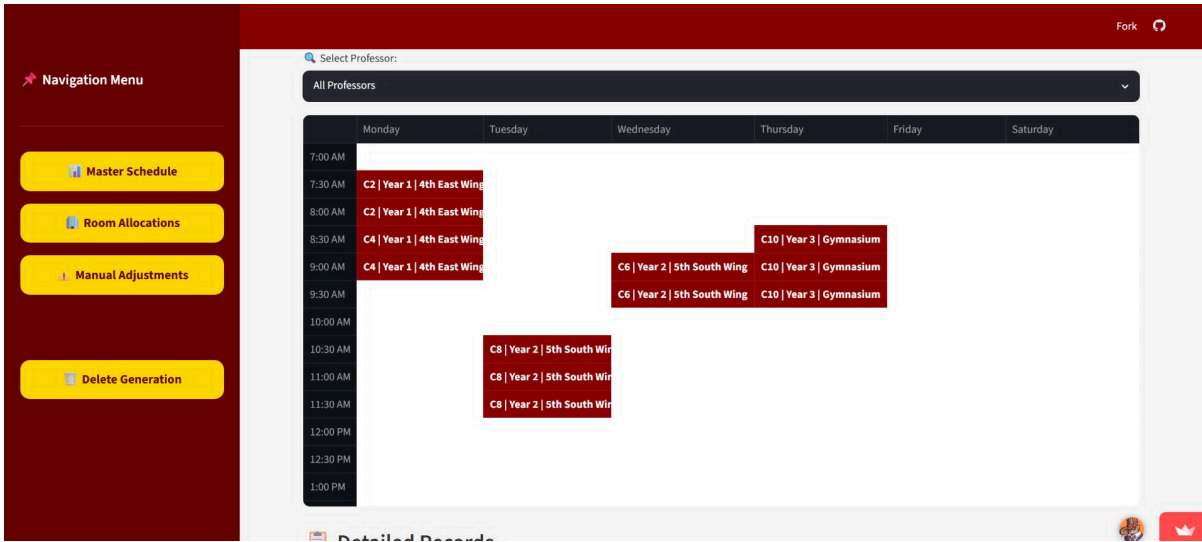
Backend	constraint_check.py	Serves as the validation engine that evaluates the schedule against strict operational rules and boundaries.	Assignment, Professor, Student, no_modality_mix, no_double_booking_professor, no_double_booking_room, lab_must_be_f2f, valid_time_bounds, valid_location, no_class_sunday, time_overlaps,
	csp_msv.py	Implements the constraint satisfaction problem and most constrained value algorithms to generate a valid base block timeline.	Course, Professor, Room, Assignment, heapq
	data_struct.py	Contains the core object oriented data structures that represent academic entities like courses, professors, students, and blocks to simulate interactions accurately.	dataclass, field, List, Optional, datetime
	database_acces.py	Manages the information pipeline by extracting raw constraints and exporting compiled results between the backend and the files.	pandas, openpyxl, gspread, sqlite3
	ga_sa.py	Executes the hybrid optimization loop combining genetic algorithm techniques and simulated annealing steps to resolve schedule quality.	DEAP, simanneal, numpy
	main.py	The main execution entry point where all independent algorithm scripts and data structures converge to	sys, Path, pandas, load_all, MasterList, Assignment, build_csp,

		run the scheduling system.	build_initial_schedule, refine_schedule, run_checker
Database	inputSheet.xlsx	The tabular spreadsheet file that holds raw university data collected from user setups and database records.	N/A
	masterSchedule.xlsx	The final compiled spreadsheet matrix that stores the completed timetable grid after algorithmic resolution is finished.	N/A
Frontend	ui.py	Renders the complete web interface using a lightweight python framework to gather configurations, showcase visual grids, and display the final outputs.	streamlit, plotly, pandas
Host	Streamlit Cloud	The cloud deployment platform that automatically pulls the files from GitHub to make the program accessible as a web application.	N/A

Security and Development Measures

NAME	DESCRIPTION
Access	Restricts system entry exclusively to authorized administrative personnel and designated college staff members through secure authentication controls to protect sensitive scheduling data.
Editing	Permits only verified administrators to modify core constraints, update database entries, and alter generated schedules while preventing general users from making unauthorized changes.
Error Resolution	Utilizes automated constraint validation checks and exception logging to quickly identify operational schedule conflicts or data entry discrepancies for

	immediate correction.
Upscaling	Employs a highly modular architectural layout allowing future developers to easily integrate more complex optimization algorithms, support larger student populations, and expand storage capabilities.



Reference

Local

- Agoylo, J., et al. (2025). Web-based scheduling system using genetic algorithm with facial recognition. <https://www.researchgate.net/profile/Jose-Agoylo-Jr-2/publication/390909500>
- Ballera, M., & Napalit, R. (2023). Optimizing a schedule using firefly algorithm with constraint satisfaction. AIP Conference Proceedings, 2508(1), 020027. <https://doi.org/10.1063/5.0160462>
- Bulawat, A., & Pena, R. (2025). Development of an automated scheduling system for NEUST Papaya off-campus: A partial ADDIE model approach. <https://www.researchgate.net/publication/392135793>

- Camungao, R. Q. (2025). A new class scheduling approach using genetic algorithm methods for Isabela State University. *Journal of Information Systems Engineering and Management*, 10(49s). <https://jisem-journal.com/index.php/journal/article/download/9909/4570/16547>
- Commission on Higher Education. (2022). *CHED Memorandum Order No. 06, series of 2022: Sustaining flexible learning in higher education: An addendum to CMO No. 4, series of 2020*. <https://ofa.upd.edu.ph/wp-content/uploads/2022/06/CMO-NO-.6-S.-2022.pdf>
- Contillo, G. L., & Querubin, A. N. (2024). Web based course scheduling system using greedy algorithm. *IEEE Xplore*. <https://ieeexplore.ieee.org/abstract/document/10809509>
- Keh, A. M., & Sarmiento, J. R. (2025). Automated class scheduling system for Aemilianum College Inc. *Global Scientific Journals*, 13(2), 545. <https://www.globalscientificjournal.com>
- Mallari, C. B., San Juan, J. L., & Li, R. (2023). The university coursework timetabling problem: An optimization approach to synchronizing course calendars. *Computers & Industrial Engineering*, 184, 109561. <https://doi.org/10.1016/j.cie.2023.109561>
- Polytechnic University of the Philippines. (2019). *Student handbook*. Office of the Student Services. <https://www.pup.edu.ph/studentservices/oss/>
- Romaguera, D., Plender-Nabas, J., Matias, J., & Austero, L. (2024). Development of a web-based course timetabling system based on an enhanced genetic algorithm. *Procedia Computer Science*, 234, 1714–1721. <https://doi.org/10.1016/j.procs.2024.03.162>
- Yee, B. E. D., Bagorio, B. I., Cabo, J., & Garbes, M. V. (2025). Interactive timetable scheduling matrix for the City College of Angeles. *Philippine Journal of Interdisciplinary Research*. <https://cca.edu.ph/assets/images/11-interactive%20time%20scheduling%20matrix.pdf>

International

- Bacchus, F., & van Run, P. (1995). Dynamic variable ordering in CSPs. In U. Montanari & F. Rossi (Eds.), *Principles and Practice of Constraint Programming — CP 1995. Lecture Notes in Computer Science*, vol. 976 (pp. 258–275). Springer. https://doi.org/10.1007/3-540-60299-2_16
- Bai, B., Dai, H., Zhang, D. J., Zhang, F., & Hu, H. (2022). The impacts of algorithmic work assignment on fairness perceptions and productivity: Evidence from field experiments. *Manufacturing & Service Operations Management*, 24(6), 3060–3078. <https://doi.org/10.1287/msom.2022.1120>
- Bangquiao, D., et al. (2023). Time management strategies of working students. *Global Scientific Journal*. https://www.globalscientificjournal.com/researchpaper/Time_Management_Strategies_of_Working_Students.pdf
- Ceccarini, C., Mirri, S., Salomoni, P., & Prandi, C. (2021). On exploiting data visualization and IoT for increasing sustainability and safety in a smart campus. *Mobile Networks and Applications*, 26(5), 2066–2075. <https://doi.org/10.1007/s11036-021-01742-4>
- Holili, M., Shafa, M. F., Widat, F., Listrianti, F., & Walid, A. (2024). Improving the quality of student learning through time management training: An experimental research. *Educazione: Journal of Education and Learning*, 1(2), 91–101. <https://doi.org/10.61987/educazione.v1i2.501>
- Lagyo, E. T. A., Galicia, J. K. A., & Guico, M. L. (2025). Design of a mobile navigation system for a university pedestrian. *Archium ATENEO*. <https://archium.ateneo.edu/cgi/viewcontent.cgi?article=1197&context=ecce-faculty-pubs>
- Landon, T. J., & Pope, N. G. (2025). Schedule-driven productivity: Evidence from nontraditional school calendars. *Journal of Policy Analysis and Management*, 45, e70075. <https://doi.org/10.1002/pam.70075>
- Lu, G., Du, R. Y., & Peng, X. D. (2022). The impact of schedule consistency on shift worker productivity: An empirical investigation.

- Manufacturing & Service Operations Management, 24(5), 2780–2796.
<https://doi.org/10.1287/msom.2022.1132>
- Ma, J. (2025). Augmented reality for campus wayfinding: Enhancing navigation efficiency and student social engagement — A case study of Leeds University Union. *Visible Language*, 59(3), 402–433.
<https://journals.uc.edu/index.php/vl/article/view/8857>
- Han, X., & Wang, D. (2025). Gradual optimization of university course scheduling problem using genetic algorithm and dynamic programming (POGA-DP). *Algorithms*, 18(3), 158.
<https://doi.org/10.3390/a18030158>
- Murad, S., et al. (2022). Optimized Min-Min task scheduling algorithm for scientific workflows in a cloud environment.
<https://www.researchgate.net/publication/358461191>
- Ortiz-Bayliss, J. C., Amaya, I., Conant-Pablos, S. E., & Terashima-Marín, H. (2018). Exploring the impact of early decisions in variable ordering for constraint satisfaction problems. *Complexity*, 2018, Article 6103726.
<https://doi.org/10.1155/2018/6103726>
- Russell, S., & Norvig, P. (2022). *Artificial intelligence: A modern approach* (4th ed.). Pearson. ISBN: 978-0-13-461099-3
- Soares, M., et al. (2023). Time management: What do university students think about it?
<https://www.researchgate.net/publication/372038295>
- Sylejmani, K., Gashi, E., & Ymeri, A. (2023). Simulated annealing with penalization for university course timetabling. *Journal of Scheduling*, 26, 497–517. <https://doi.org/10.1007/s10951-022-00747-5>
- World Bank. (2022). Digital transformation of Philippine higher education. World Bank Group. <https://doi.org/10.1596/38481>
- Zhang, Y., Yip, C., Lu, E., Chen, L., Guo, Y., Wang, S., Zhong, Y., Kwan, T., Lee, T., & Li, J. (2022). A systematic review on technologies and applications in smart campus: A human-centered case study. *IEEE Access*, 10, 16134–16149.
<https://doi.org/10.1109/ACCESS.2021.3053836>